

Using the chr() and ord() Functions

Python's built-in `chr()` function accepts an integer argument and returns a string of the number's character. The `ord()` function does the opposite: it accepts a string argument of a single character and returns the character's number. This number is the code point for the character in the Unicode standard.

For example, enter the following into the interactive shell:

```
>>> chr(65)
'A'
>>> ord('A')
65
>>> chr(66)
'B'
>>> chr(9829)
'♥'
```

Not all numbers are valid code points for printable characters. The terminal windows that show the text output of programs may be limited in which characters they can display. The font the terminal window uses also must support the character your program prints. The terminal window prints a Unicode replacement character, `◆`, for any character it is unable to print.

Windows' terminal window has a far more limited range of characters it can display. This set is known as the Windows Glyph List 4, and it appears in this appendix and on Wikipedia at https://en.wikipedia.org/wiki/Windows_Glyph_List_4.

The code points for characters are often listed with a base-16 hexadecimal number, as opposed to the base-10 decimal numbers we are used to. Instead of the decimal digits 0 to 9, hexadecimal has the digits 0 to 9 and then continues with the letters A to F. Hexadecimal numbers are often written with a `0x` prefix to denote that the number that follows is in hex.

You can convert a decimal integer value to a string of the hexadecimal number with the `hex()` function. You can convert a string of the hexadecimal number to a decimal integer with the `int()` function, passing 16 as the second argument. For example, enter the following into the interactive shell:

```
>>> hex(9)
'0x9'
>>> hex(10)
'0xa'
>>> hex(15)
'0xf'
>>> hex(16)
'0x10'
>>> hex(17)
'0x11'
>>> int('0x11', 16)
17
```
